Core Python

Introduction

Programming languages allow us to give our computers instructions to perform a desired task. In EECS 16A, we'll often be dealing with Python, which is a high-level language, meaning that it's closer to English than to zeroes and ones.

We can think of Python as a versatile, multipurpose calculator. In fact, if you have Python installed on your personal computer, you can use it in command-line to evaluate arithmetic expressions:

```
Python 3.8.5 (default, Sep  3 2020, 21:29:08) [MSC v.1916 64 bit (AMD64)] :: Anaconda, Inc. on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> (8*5)/4+7-1
16.0
>>>
```

Variables Assignment

Ok, so having a calculator is nice and all, but if we wanted to just do simple calculations, we'd just use the calculator app. One advantage of programming languages is that we can define *variables*, elements of a program that can *change*.

In Python, variable names can be any sequence of letters (a-z, A-Z), numbers (0-9), or underscores (_) as long as they don't begin with a number. Assigning a value to a variable is relatively straightforward: we simply use the equals (=) sign. After assigning a value to a variable, we can use the variable name to perform our desired task:

```
>>> eecs16a_rocks = 8*8
>>> eecs16a_rocks / 4
16.0
```

One important note with assigning variables is that the *entire* right side of the equals sign is evaluated before assignment. This is a convenient property because it allows us to do cool things like increment variables (add one regardless of current value):

```
>>> eecs16a_rocks = 15
>>> eecs16a_rocks = eecs16a_rocks + 1
>>> eecs16a_rocks
16
```

In fact, modifying variables based on their current values is so common that the folks who made Python gave us a shortcut. Instead of saying **var = var + 1**, we can simply say **var += 1**. Other arithmetic operations are shortened similarly:

```
>>> eecs16a_rocks = 15
>>> eecs16a_rocks += 1
>>> eecs16a_rocks
16
>>> eecs16a_rocks = 8
>>> eecs16a_rocks *= 2
>>> eecs16a_rocks
16
>>> eecs16a_rocks = 2
>>> eecs16a_rocks **= 4
>>> eecs16a_rocks
16
```

Note that exponentiation is denoted by a double asterisk (**).

Data Types

A “data type” is a class of *information*. In daily life, we deal with many data types without thinking about them! For example, if you ask your TA how many labs there are in EECS 16A, you would expect an integer as her response. On the other hand, if you wanted to know your score on a particular midterm as a percent, you’d probably expect a decimal value (which is called a ‘float’ in Python-speak).

Aside from numbers, there are many other data types as well. For example, much of our communication comes in the form of words, which we can abstractly define as a sequence of characters. These are known as ‘strings’ in programming languages. Another very important data type is the ‘boolean.’ These are special types that can only take on two values: True and False. Continuing with our analogies of data types to daily

life, a boolean value might represent what you'd expect to hear when you ask a yes-or-no question.

Luckily for us, Python handles a lot of this stuff behind the scenes. That's why we don't have to provide any information about the data type when entering **eecs16a_rocks = 16** -- Python automatically decides to store **eecs16a_rocks** as an integer.

Let's look at a few examples of variable declarations and manipulations for different data types.

Strings

```
>>> eecs16a = "rocks!"
>>> eecs16a * 2
'rocks!rocks!'
>>> eecs16a[1]
'o'
```

In this example, note how we input strings surrounded by quotation marks. Python doesn't care whether you use single quotes (‘) or double quotes (“), but please pick one and stick with it. You can't start a string with a single quote and end with a double quote (sadly, Python isn't that smart).

Additionally, the multiplication operator is defined to duplicate the string a certain number of times. This is an example of how we can redefine operations to work over different data types. However, other arithmetic operators are not defined by default for strings.

Finally, we can “pick out” the character at a given index of a string using the square bracket ([index]) notation in the example. IMPORTANT: the index begins at 0, not 1, which is why asking for the character at position 1 gives the second character in the example above.

We can also add string together (what's called *concatenation*) using the addition operator:

```
>>> "eecs16a" + " rocks!"
'eecs16a rocks!'
```

Booleans

```
>>> eecs16a_rocks = True
>>> eecs16a_rocks
True
>>> not eecs16a_rocks
False
```

A boolean is either **True** or **False**. Note the capitalization -- **Python is case-sensitive!** As you can see in the example, we can negate a boolean in Python with the **not** operator. However, booleans aren't super useful by themselves. Instead, we often *produce* booleans by comparing things. For example, the expression **var1 > var2** will spit out a boolean value depending on whether **var1** is greater than **var2**. Similarly, we can check "less than" with **<** and "equality" with **==**. For example,[1]

```
>>> cal_2019_score = 24
>>> stanfurd_2019_score = 20
>>> cal_2019_score < stanfurd_2019_score
False
>>> cal_2019_score == stanfurd_2019_score
False
>>> cal_2019_score > stanfurd_2019_score
True
```

Sometimes we want to create more complex boolean expressions using logical operators (such as **and** or **or**):

```
>>> cal_2019_score == stanfurd_2019_score or cal_2019_score > stanfurd_2019_score
True
```

This above will check if the scores are equal OR if the Cal score is greater than the Stanfurd score.

Lists

This one's a big one. Lists are a way for us to store a bunch of values. Formally, lists in Python represent an ordered collection of data, and (unlike some languages) we're not

[1] Disclaimer: no cardinals were harmed in the making of this example.

restricted to storing data of the same type. However, in this class, most of the lists you'll be manipulating will contain values of the same data type.

We can create lists using square bracket notation:

```
>>> primes_1_to_20 = [2,3,5,7,11,13,17,19]
>>> primes_1_to_20[5]
13
```

Notice how we access values within the list. We simply request for the value at index 5 using more square brackets. IMPORTANT: Python is what's called a zero-indexed language, meaning that the list starts at index 0. That's why we're getting the sixth element of the list despite asking for index 5. Notice how similar this is to string manipulation -- in many languages, strings are internally stored as lists of characters.

A very useful tool Python gives us is called a *list comprehension*. This allows us to build a list using the elements from another list. More formally, we execute an operation on the list element-wise to produce a new list. A list comprehension takes the following form:
[operation(var) for var in old_list]

For example, let's say we wanted to create a list whose values are the squares of the first eight prime numbers. Since **primes_1_to_20** contains the first eight prime numbers, we want to simply square each element to produce a new list. We can do this with a list comprehension:

```
>>> squares_of_primes = [prime**2 for prime in primes_1_to_20]
>>> squares_of_primes
[4, 9, 25, 49, 121, 169, 289, 361]
```

Note that in the list comprehension, we chose **prime** as our variable name. This was arbitrary -- it could have been named anything -- but it was chosen because it's descriptive.

Values in lists are *not* permanent. They can be changed using their indices. For example,

```
>>> list_of_good_opinions = ["pineapple", "pizza", "is", "a", "culinary", "disaster"]
```

Oh, no! Someone obviously made a mistake. We can fix this by indexing into the list and changing an element:

```
>>> list_of_good_opinions[5] = "masterpiece"
>>> list_of_good_opinions
['pineapple', 'pizza', 'is', 'a', 'culinary', 'masterpiece']
```

There we go! All better.

Note that for the array above, there was already a value stored at index 5. When we changed the element, we replaced the value at index 5. However, if we had only five elements in our array, there would be no index 5.

```
>>> list_of_good_options = ['pineapple', 'pizza', 'is', 'a', 'culinary']
```

To add another element to a list, we **append** it to the list. For example, if we wanted to add ‘masterpiece’ to match the list above,

```
>>> list_of_good_options.append('masterpiece')
>>> list_of_good_options
['pineapple', 'pizza', 'is', 'a', 'culinary', 'masterpiece']
```

Defining Functions

There are two ways to define a function, the first is the def statement and the second is a lambda function. You want to define a function for repeated use later on without writing the same code again. Quick example, if you are trying to do a series of vector addition, it’s better to write the procedure into a function, instead of repeating the same steps multiple times.

def

To use **def** to define a function, you write

def function_name(variables your function takes):
    operations
    …

To call your defined function, you write **function_name(variables)**

Let's look at an example.

```
x = 1
x = x**2 + 1
lst = [1,2,3,4,5,6,7,8,9,10]
```

We have **x**, and we squared it and then added 1 to it.
Now, we want to do the square and add 1 operation to all the numbers in **lst**. Instead of writing the same **x**2**, we can define a function that carries out this operation.

```
def square_add_one(x):
   return x**2 + 1
```

Note, this **x** has no particular meaning and has nothing to do with the **x** we've defined before.
Now, let's try to execute our function, we will see a 5 being printed.

```
y = square_add_one(2)
print(y)
```

Now, to actually apply this to all of lst's elements, we can use a loop.

```
for i in range(len(lst)):
   lst[i] = square_add_one(lst[i])
print(lst)
```

We get **[2, 5, 10, 17, 26, 37, 50, 65, 82, 101]**

return

Return marks the end of your function. Nothing you write after the return statement will matter because they won't be executed. Returned values need to be bound to something, otherwise they will be lost. For example,

```
square_add_one(2)
```

will return 5, but since it's not bound to any variable, its value, 5, will be lost.

Note, functions don't all need to contain return statements, but return statements can only exist in functions. A function without a return statement will return a value of **None** (this is also called null or nil in other programming languages). For example, returning in a loop outside of any functions will error:

```
for i in range(len(lst)):
```

```
    return square_add_one(lst[i])
```

For an example of a function that doesn't return anything (i.e. **None**):

```
def no_return(x):
    for i in range(len(x)):
        x[i] = 0
```

This function doesn't return anything, but by calling this function on a certain list, it will change all of its elements to 0 (do you see why?). So functions without any return statement can still make an impact. Or, you can also make meaningless functions if you so desire.

```
def why_am_i_here():
    i = 0
```

Although this function does nothing, there's something interesting that it demonstrates, it shows that functions can also take no arguments. For example, the function **change** can manipulate the list **x** without taking in arguments:

```
x =[]
def change():
    x.append(1)
change()
print(x)
```

We will see **[1]**, instead of **[]**, as a result of calling change.

Optional inputs

You can also write a function in a way that it can take an arbitrary number of arguments, with corresponding default values if nothing is given. You can assign these values by using the equal sign. Note, you can't have any non default variables after default variables, so write down variables you're certain will be given first.

```
def add_one_or_x(y, x = 1):
    return x + y
print(add_one_or_x(2))
print(add_one_or_x(2,4))
```

This will print out 3 and 6 respectively.

It goes without saying that you can also have a function taking only default variables.

```
def auto(x=1,y=1):
   return x*y
print(auto())
```

This will output 1*1 = 1

Lambda

Lambda is another way of defining a function. For the purpose of the class it has little importance, so no worries if you don't understand anything even after looking at this doc, but lambda is a very useful tool for coding in general (and especially in 61a). Lambda essentially does what def does, but in one line.

To write a lambda function, you will:
lambda x,y,z… : operations…

```
lambda_function = lambda x: x + 1
i = lambda_function(1)
print(i)
```

Carrying out these lines of codes, you'll get a 2, because this lambda expression takes in a variable, and returns it, plus 1, and 1 + 1 = 2.

Look at the syntax, it's '**lambda x: an expression**', **x** is the same as variables a def function takes, and the expression is what you're returning. Different from a typical def function, a lambda function can only return.

So, like a def function, lambda can also take multiple variables, or no variable at all.
See if you can work out what will be output.

```
lambda2 = lambda x,y: x + y
print(lambda2(2,3))
lambda3 = lambda : 'nothing'
print(lambda3())
```

The answer is 5, and nothing, can you see why?

Another bonus for lambda expressions is that it can be defined and called instantly, like a single use function that will be forgotten after the line it's defined and called in.

```
print((lambda x: x+1)(1))
```

Again, this outputs 2.

A short summary of **lambda** compared to **def**, lambda can be written on the fly, but can only execute return statements

Higher Order

Again, you won't be needing this knowledge for this class in particular, so no worries if you don't understand, or just skip this entirely.

Since there's no restrictions on what can be used as input variables, you can also input functions into functions, and such a function is called higher order function. Look at this function that takes **f** and **g**, and returns another function that upon receiving an input, will return **f(g(input))**.

```
def higher_order(f,g):
   def return_function(x):
       return f(g(x))
   return return_function
square_add_one2 = higher_order(lambda x: x**2, lambda x: x+1)
print(square_add_one2(2))
```

This will output 9, let's break down what's going on.

Look at **higher_order**, it takes two arguments, and returns another function that will return **f(g(x))** when giving an input **x**. Note, it returns such a function, and not any particular number, to get a concrete return like 9, you need to call the returned function again on something.

Look at the next line, we have **lambda x: x**2** and **lambda x: x+1** as inputs, this is coincidentally a demonstration of the benefit of using lambda. If we're not using lambda, we need to spend way more lines of code to define a square and an add one function, but with lambda both are done in one line. So, our composed function will first add one to whatever input it was given, and then square it, and (2+1)^2 = 9.

Lastly, do you see that **return_function** is using variables not particularly its own? Why this works be discussed later (or sooner? depending on arrangements) in the environment diagram sections of the guide.

Can this be done using only lambda?

The answer is yes, since we're only doing returns here. Can you think of how to do this?

```
higher_order_lambda = lambda f,g: lambda x: f(g(x))
square_add_one2 = higher_order_lambda(lambda x: x**2, lambda x: x+1)
print(square_add_one2(2))
```

Do you feel the power of lambda expressions?

Python Functions & Documentation

print()

You print variables or primitive data types, useful for debugging. Caution: **print(x)** is not the same as **print('x')**, the former, will print out whatever is bounded to the variable x (if it exists, error otherwise) while the later will print out x, just x, as a letter.

```
print('e'+'e'+'c'+'s'+'16'+'a')
```

Note, strings can be added up using +, this will print out **eecs16a**
Print can also take multiple arguments, and they will be separated by spaces.

```
print('e','e','c','s','1','6','a')
```

This will print **e e c s 1 6 a**

type()

Returns the data type a certain variable is, can only take one argument.

```
print(type(1))
print(type('x'))
print(type(1)==type(1.0))
```

The output will be **<class 'int'>**, **<class 'str'>**, and **False**, because 1 is an int, while 1.0 is a double.

abs()

Returns the absolute value of input, can only take one argument

pow()

Takes two arguments x and y and returns **x^y**

sorted()

A very multi use function, most general use is to sort lists or strings, by default it's in ascending order, but you can change that by adding **reverse = True**.

```
print(sorted('eecs'))
print(sorted([3,4,1,2]))
print(sorted([3,4,1,2], reverse = True))
```

Note, using sorted on strings will create a list, which can be turned back into a string by using **join**. **x.join(list)** will join all elements of a list into a string by inserting x in between them

So the output will be **['c', 'e', 'e', 's']**, **[1,2,3,4]**, and **[4,3,2,1]**.

```
x = ''.join(sorted('eecs'))
```

And **x** will be **cees**, because you're joining with '', an empty string. You can also use different things to join, for example.

```
x = ','.join(sorted('eecs'))
```

And x will become **c,e,e,s**

Lastly, **sorted** can also be abbreviated into **lst.sort()**

max(), min()

Pretty self explanatory. Note, you can also input a list into this to return the max/ min element of a list.

```
x = max([1,2,3,7,5])
```

And x will be 7

Len()

Gets the length of lists or strings, takes one argument.

Sum()

Returns the sum of all the elements in a list, with an optional second argument as an integer that also adds to the total sum.

```
print(sum([1,2,3,4]))
```

```
print(sum([1,2,3,4],5))
```

This will be 10 and 15 respectively

Debugging

Introduction

As you write code, you will eventually run into errors. Sometimes, it can be as small as a typo or as big as requiring a significant rewrite. In this section, we will go over common errors that you may encounter as well as how to figure out what Python is complaining about.

Common Errors

NameError: name '___' is not defined

Chances are, a variable that you are trying to use or a function you are calling is misspelled. Python cannot find the symbol that you are referencing.

If you are getting 'np' is not defined, you might have missed running the code block that imports Numpy. Look for the code block with **import numpy as np**! Remember that you have to re-run this every time you restart the kernel or re-open your notebook.

IndexError: list index out of range

Double check how you are indexing into elements in a list / Numpy array! You cannot use an index that is longer than the list itself (e.g., **list[5]** when the list only has 3 elements). If you have a loop, double check your loop!

SyntaxError: invalid syntax

Most likely a typo somewhere… Double check that your code is following Python syntax!

IndentationError: expected an indented block

Recall that you need indents after def or a loop!

ValueError: cannot reshape array of size _ into shape (___)

This is most likely caused by an error in calling np.reshape. Double check that the array size matches the new shape - they must have the same number of elements.

ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0…

This is most likely caused by dot-multiplying two matrices (e.g. np.dot) with the wrong sizes. Recall that when you are trying to multiply two matrices, the number of columns in the first matrix needs to match the number of rows in the second matrix. Numpy will error if they don't match!

ValueError: all the input array dimensions for the concatenation axis must match exactly…

When you concatenate numpy arrays (e.g. np.concatenate, np.hstack, np.vstack), make sure that the appropriate dimensions of the arrays are equal. If they don't match, try to reshape them into their appropriate sizes first!

Debugging Methods

A lot of times, life isn't that simple. If you get an error that doesn't match any of the ones we listed above, there are also many other ways for you to debug your code.

Print Statements

One of the simplest debugging techniques is inserting print statements throughout your code to print variables or markers. As your code runs you can monitor how many times a for loop has run or how variables have changed. Before you print, you should think of the expected outcome of your print statements and then carefully follow each printed statement to see where they don't match. Inserting lots of print statements can help you narrow down what portion of your code isn't behaving as expected. This method is especially useful when using any sort of loops or self-defined functions.

Printing can also be helpful when exploring what predefined functions/variables return. This can help you figure out if you are using them correctly.

Useful Resources

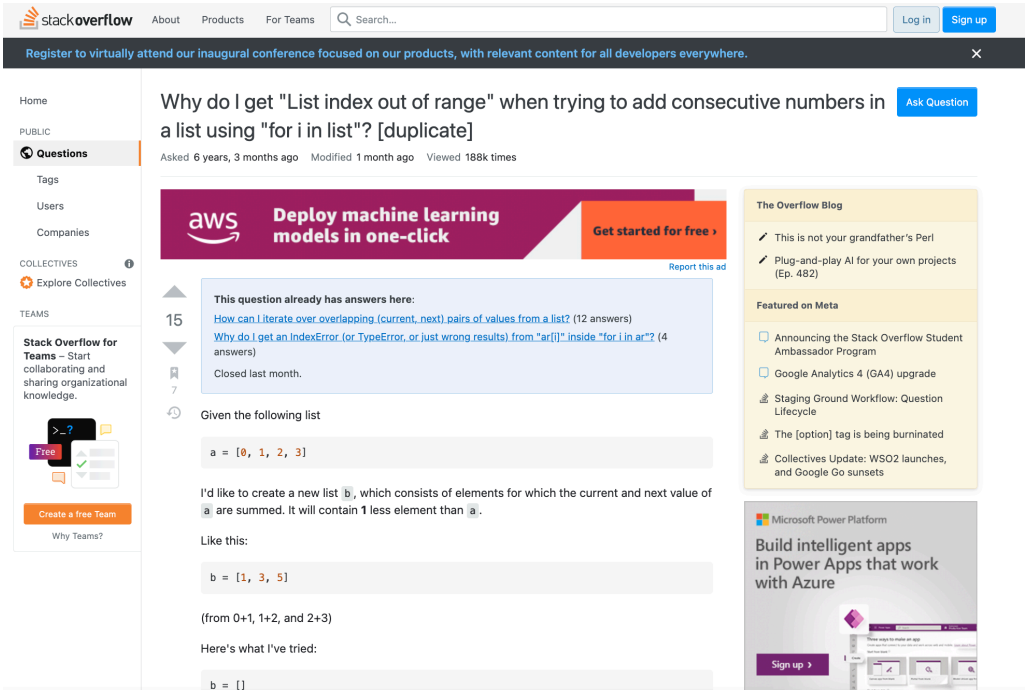
Internet Search

A simple search on the web using the right keywords can be incredibly helpful in finding useful resources such as documentation or answers to questions related to debugging. Include the

language you are working and relevant information such as error codes, or functions you are using.

Programmers often joke that Googling is the number one skill needed to program.

StackOverflow

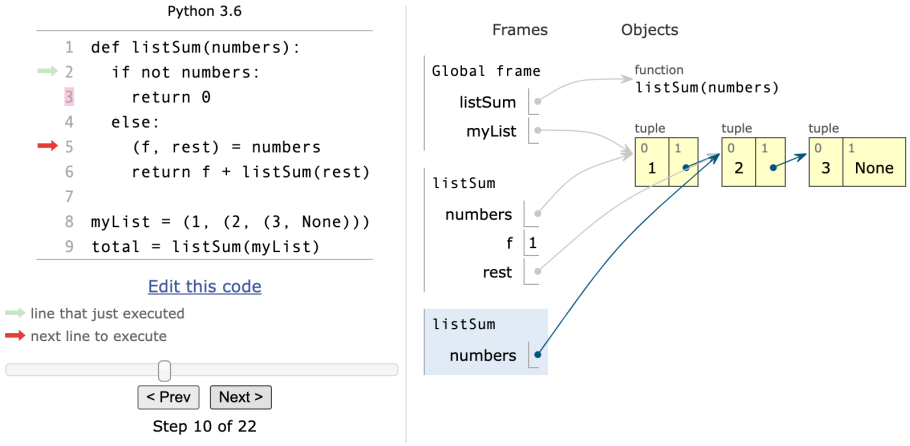


StackOverflow is a forum to ask and answer questions related to programming which may be useful for debugging. Think Quora or Yahoo Answers but for programmers. It's easy to find questions you may have that are already answered. Sometimes when documentation fails, StackOverflow is the next best option for unique syntax related questions.

Python Tutor

Python Tutor is a really helpful tool introduced to CS61A students which can help you visualize a program's execution step by step. Simply input your code into python tutor and take a tour of your code's execution line by line. Python tutor will keep track of variables and function calls. This tool is helpful for more complicated programs introduced in 61A, but may be helpful for you as you learn to use loops, self-defined functions, and lists.

Note that it will not be particularly useful for NumPy related exploration or code that relies on data from a csv, but definitely try it out when exploring pure Python.



Numpy

Introduction

All of this is drawn from lab 0.

Note, numpy is not built into python, for situations like these, you must import the package before using anything related to it.

```
import numpy as np
```

You can simplify the names of these packages, and almost always these are agreed names, so don't name your numpy package as, say nump, instead of **np**

Numpy is useful in assimilating matrices and vectors, and doing calculations associated with them, which is something that python lists really lack.

Arrays/Slicing

The first thing to note about numpy arrays is that it's NOT the same as a python list. Just because you can use some list related functions on arrays doesn't mean that they're the same type of objects.

What is true though however, is that you can create arrays from lists.

```
py_lst = [1,2,3,4]
np_arr = np.array(py_lst)
```

Similar to lists, you can slice arrays:

For example

```
arr = np.array([1,2,3,4,5,6,7,8,9,10])
print(arr[:9])
print(arr[-2:])
```

Would print **[1,2,3,4,5,6,7,8,9]** and **[9,10]**
Note, **list[-x:] = list[len(list)-x:]**

Different to lists though, is 2D array slicing.
To do this, you write **array[a:b,c:d]**, which will get the **a** to **b** rows of the **c** to **d** columns.

Note, when you see **[::2]**, this means every other element, why?
[a:b:d] means to get elements from **a** to **b**, separated by **d** each.

For example

```
arr = np.array([1,2,3,4,5,6,7,8,9,10])
print(arr[1:9:2])
```

would print **[2,4,6,8]**, because **1:9** corresponds to **[2,3,4,5,6,7,8,9]**, and we're going in 2 steps.

Looking back at **[::2]**, you can now see that this means you go from the start of the list to the end (because no index is given), 2 steps at a time.

As an example, can you work this out?

```
i = np.array(range(25), dtype=np.int).reshape([5,5])
print(i[0])
print(i[:,0])
print(i[1:4])
print(i[:,1:4])
print(i[:3,:3])
print(i[:,::2])
```

First, i is a 5x5 matrix.
i[0] is the first row

i[:,0] is the first column, because you're not specifying anything about the row, while the second 0 gets the first column
i[1:4] is the second to fourth rows
Similarly, i[:,1:4] is the second to fourth columns
i[:3,:3] is the top left 3x3 of i, can you see why?
Lastly, i[:,::2] is every other column of i, as we've mentioned before.

Functions & Plain English documentation

np.eye(N, M=None, k=0, dtype=<class 'float'>, order='C', *, like=None**)**

The eye function is useful for creating identity matrices. It takes in parameters N and M to create an NxM matrix with a diagonal of 1s. N is the only required parameter in the function listed above. If no M is submitted, M defaults to N creating a square matrix. K allows us to change the starting row of the series of 1s.

Note that unbolded parameters are optional, which means they will default to the values following the = sign.

```
np.eye(10, 6)
```

```
array([[1., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 1.],
       [0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0.]])
```

```
np.eye(5, 5)
```

```
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 1.]])
```

np.zeros(shape, *dtype=float, order='C', *, like=None***)**

The zeroes function is useful for creating matrices that are entirely zeroes. It takes in a shape to indicate the shape of the new array/matrix. It can be in the form of a tuple (3, 4) or a single number 3. We can further customize the type of the matrix contents, but that doesn't happen often in EE16A.

Note that unbolded parameters are optional, which means they will default to the values following the = sign.

```
np.zeros((2, 3))
```

```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

```
np.zeros(5)
```

```
array([0., 0., 0., 0., 0.])
```

np.ones(shape, dtype=None, order='C', *, like=None**)**

The ones function does exactly the same thing as the zeros function except it returns a matrix of ones instead of zeroes. It takes in a shape to indicate the shape of the new array/matrix. It can be in the form of a tuple (3, 4) or a single number 3.

Note that unbolded parameters are optional, which means they will default to the values following the = sign.

```
np.ones((4, 5))
```

```
array([[1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.]])
```

```
np.ones(5)
```

```
array([1., 1., 1., 1., 1.])
```

np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0**)**

Linspace allows you to create larger arrays without manually typing every value. The start indicates the first integer your array will start with and the stop parameter indicates the last. The third value, num indicates the count of the number of steps your array will take between the start and stop. Sometimes creating the right array does not come intuitively in terms of the steps vs start and stop, so when using linspace try printing your array while forming it to double check you have created the array you intended.

```
x = np.linspace(1, 9, 5)
x
```

```
array([1., 3., 5., 7., 9.])
```

```
x = np.linspace(1, 10, 5)
x
```

```
array([ 1.  ,  3.25,  5.5 ,  7.75, 10.  ])
```

np.arange(start, stop, step, dtype=None, *, like=None**)**

Arrange returns an array with the specified start, stop, and step. Note that the major difference between linspace and arange is that linspace specifies the number of steps while arange specifies the size of the steps between start and stop.

np.transpose(a, axes=None**)**

The transpose function helps us transpose a matrix, a. In other words we reverse the axes of the matrix, giving us the matrix transpose. A 3x4 matrix will become a 4x3 matrix with its rows and columns reversed as shown in the example below. Calling x.shape will show the original shape of the matrix. If we call .shape on the new matrix we will find its rows and columns have been flipped. Axes allows us to determine which axes of the matrix we want to flip on. However, when getting a transpose not passing in axes works just fine. For the most part, you will not need to provide an axes to retrieve the desired transpose.

Alternatively you could write a.T to retrieve the transpose.

```
x = np.array([[1, 3, 5, 6],
    [5, 8, 2, 9],
    [0, 5, 6, 7]])
np.transpose(x)
```

```
array([[1, 5, 0],
       [3, 8, 5],
       [5, 2, 6],
       [6, 9, 7]])
```

np.linalg.invinalg.inv(a)

Linalg.inv gives us the multiplicative inverse of a. Note that linalg.inv will error if given a matrix that has a determinant of zero.

```
x = np.array([[1, 3, 5, 6],
    [5, 8, 2, 9],
    [0, 5, 6, 7],
    [4, 2, 4, 1]])
np.linalg.inv(x)
```

```
array([[ 0.19861432,  0.06235566, -0.26789838,  0.12240185],
       [-0.62355658,  0.06004619,  0.44572748,  0.08083141],
       [ 0.00230947, -0.1039261 ,  0.11316397,  0.12933025],
       [ 0.44341801,  0.04618938, -0.27251732, -0.16859122]])
```

np.reshape(a, newshape)

Reshape will reshape an existing matrix of NxM size to a new shape that is specified. Newshape is a tuple indicating the new N and new M. The product of the new NxM must equal the product of the old NxM to preserve all the values in the matrix.

```
x = np.array([[11, 4, 5, 14],
    [7, 1, 5, 4],
    [1, 6, 4, 9],
    [4, 9, 1, 17]])
np.reshape(x, (2, 8))
```

```
array([[11,  4,  5, 14,  7,  1,  5,  4],
       [ 1,  6,  4,  9,  4,  9,  1, 17]])
```

np.dot(a, b)

The dot function gives us the dot product of two vectors or matrices, a and b. Calling a@b is another method of finding the dot product of two matrices.

```
x = np.array([[1, 4, 5, 2],
    [7, 1, 5, 4],
    [1, 6, 4, 9],
    [4, 1, 1, 1]])

y= np.array([[1, 4, 5],
    [3, 1, 5],
    [1, 6, 4],
    [4, 2, 1]])
np.dot(x, y)
```

```
array([[26, 42, 47],
       [31, 67, 64],
       [59, 52, 60],
       [12, 25, 30]])
```

Less frequently used functions:

(Not really a function,but): Different from python list, array addition is element wise.
Ex:
List: [1,2] + [3,4] = [1,2,3,4]
Array: [1,2]+[3,4] = [4,6]

x.shape: returns a 2 element list, where x.shape[0] is the number of rows and x.shape[1] is the number of columns of x. x is a Numpy array / matrix.

np.hstack & np.vstack: hstack refers to horizontal stacking of arrays. So you add columns. And vstack refers to vertical stacking of arrays, or adding rows.

np.hstack([array1, array2]), caution, see that hstack and vstack only take one argument, which is kinda two arguments, but stuffed in one list.

Note, this DOES NOT mutate the original array, and instead creates an entirely new one that will be lost if you don't bond it to a variable.

For this example, remember previously that i is 5x5

```
while i.shape[1] < 6:
   np.hstack([i,np.array([[0] for x in range(i.shape[0])])])
```

This will cause an infinite loop.

```
while i.shape[1] < 6:
   i = np.hstack([i,np.array([[0] for x in range(i.shape[0])])])
```

This won't, do you realize why?

Again, hstack creates a new array, so i is never changed in the first case, but in the second case, by bonding the returning new array to i, the loop will terminate after one execution of hstack.

np.concatenate([array1…arrayn], axis = 1 or 0), this is essentially hstack and vstack. So the same stuff with hstack and vstack applies (doesn't mutate original array, takes all arrays as one argument, etc) axis = 0 equals vstack, and 1 equals hstack, default argument of axis (so you don't write axis =) is 0.

np.argmax(array), np.argmin(array): returns the INDEX of the maximum or minimum entry. Ex: argmax([1,2,5,4,11,4]) = 4.

And like that, you've read through most of the numpy stuff you need for the class. Don't sweat if you can't remember them, even if you do now, we're pretty sure every time you need to use these functions you'll have to google them up again, and that's absolutely FINE.

As a reward for reading this intro guide, here's Shuming's (one of the past TAs who greatly contributed to making this guide) dog eating a taco :D